Which Parts of the "Modern Transformer Recipe" Actually Matter? A Controlled Ablation on TinyStories

Cohen-Transformer

June 2026

Abstract

Contemporary language-model training stacks bundle a long list of small architectural and optimization tweaks — Muon, QK-norm, learnable QK-gain, LayerScale, value residuals, z-loss, logit softcapping — and present them as a single "modern recipe." We disentangle this bundle with a controlled, multi-seed ablation. A fixed ∼97.6M-parameter decoder-only transformer is trained on TinyStories under an identical token budget (30.72M tokens, 1000 steps) across 22 configurations × 3 seeds = 66 runs, structured as a fractional factorial design (all-off and all-on anchors, single-feature add-ins, leave-one-out drops from the full recipe, and targeted 2×2 interaction cells). Two components dominate: switching AdamW → Muon and adding QK-norm each cut final validation loss by ≈0.13 nats, and the two are strongly *synergistic* — together they buy ≈0.24 nats, far more than the sum of their individual effects. LayerScale and value residuals each contribute a smaller but real ≈0.05 nats and are mildly synergistic with each other. QK-gain and logit softcap are effectively free riders ($|\Delta| < 0.002$ nats), and z-loss is mildly harmful at this scale (+0.016 nats). The best single configuration is in fact the full Muon recipe *minus* z-loss (1.651 val loss), not the kitchen-sink "everything on" (1.669). All 66 runs trained to completion with zero divergences.

1 Introduction

The recipes shared in open LLM-training projects are cumulative: each new trick is added on top of the last, and the resulting stack is evaluated as a whole. This makes it hard to know which ingredients carry the gains, which are inert, and which are actively counterproductive at a given scale. It also makes the recipes hard to port — a practitioner copying the full stack inherits its dead weight and its failure modes.

This paper takes the seven toggles in the `Cohen-Transformer` training stack and measures each one in isolation and in combination, holding everything else fixed (model size, data, token budget, learning-rate schedule, weight decay, gradient clipping, batch order). The question is deliberately narrow and answerable: *under a fixed compute budget, what does each component contribute to validation loss, and do the components interact?*

2 The components under study

All seven are independent toggles over a common backbone (12 layers, 12 heads, $d_{\text{model}} = 768$, SwiGLU MLP, RMSNorm pre-norm, RoPE, untied LM head). The backbone with all toggles off is the AdamW baseline.

Toggle	What it does	Where
optimizer	Muon (Newton–Schulz-orthogonalized momentum, `match_rms_adamw` LR) on 2D body matrices + AdamW on embeddings/head/scales, vs. pure AdamW	optimizer
qk_norm	Per-head RMSNorm on Q and K before RoPE (Gemma-2 style)	attention
qk_gain	Learnable per-head multiplicative gain folded into Q (scales scores pre-softmax)	attention
layerscale	Learnable per-channel residual gates on the attention and MLP branches (CaiT), init 0.1	residual
value_residual	Each layer mixes its value with layer-0's value via a learnable per-layer λ (ResFormer, arXiv:2410.17897)	attention
z_loss	`lse_square_scale` = 1e−4 regularizer on the log-partition	loss
softcap	tanh logit softcap at 30.0	loss

z-loss and softcap are loss-side options of `LigerFusedLinearCrossEntropyLoss`; the rest are model/optimizer changes. The Muon + AdamW split and weight-decay policy match the production `simple.py` stack.

3 Experimental setup

Model. ∼97.6M parameters (the small variation across configurations — 97,536,768 to 97,573,787 — is the handful of extra scale/gate parameters some toggles add; it is <0.04% and does not confound the comparison).

Data. `roneneldan/TinyStories`, tokenized once with a fixed 8192-token byte-level BPE into a memory-mapped `uint16` corpus. Every run reads identical (x, y) blocks in identical order (no shuffling, deterministic batches), so the only differences between runs are the toggles and the seed.

Budget. 1000 optimizer steps, batch size 15, gradient accumulation 1, block size 2048 → 30,720 tokens/step → **30.72M tokens per run**. LR peak 1e−3, 99-step linear warmup then cosine decay to 0.1× peak, weight decay 0.1, gradient-norm clip 1.0, bf16 autocast, `torch.compile`. Validation loss is the mean cross-entropy over 100 held-out blocks at the final step.

Design. A full 2^7 grid (128 configs) was not run. Instead a fractional factorial focuses the budget on the questions that matter:

- **Anchors** — all-off (pure AdamW baseline) and all-on (full Muon recipe).
- **Add-one** — baseline + exactly one toggle, for clean main effects of the features expected to matter on their own.
- **Leave-one-out** — full recipe with exactly one toggle removed, to measure each component's *marginal* contribution inside the complete stack.
- **Targeted 2 × 2 cells** — Muon×QK-norm, LayerScale×value-residual, z-loss×softcap, z-loss×QK-gain, softcap×QK-gain, for interaction terms.

22 unique configs × 3 seeds = 66 runs. Main effects are computed over matched on/off pairs (configs identical except the factor in question); interactions over matched 2 × 2 quadruples. Effects are reported as Δ(final val loss) = ON − OFF, so **negative means the component helps**.

4 Results

4.1 Main effects

Factor	mean Δ	std	n pairs	verdict
qk_norm	−0.128	0.070	9	large help
optimizer (Muon)	−0.128	0.079	9	large help
value_residual	−0.053	0.032	9	moderate help
layerscale	−0.051	0.034	9	moderate help
qk_gain	−0.002	0.008	15	negligible
softcap	−0.000	0.010	15	negligible
z_loss	+0.016	0.008	15	mildly harmful

Two findings stand out. First, QK-norm and the Muon optimizer are the workhorses of the recipe, each worth ≈0.13 nats — an order of magnitude more than the next tier. Second, **z-loss consistently hurts** at this scale: its effect is positive in mean and tight in variance (std 0.008 over 15 pairs), so this is a real signal, not noise. z-loss is a numerical-stability regularizer designed for large-vocab, long-schedule training; with an 8192-token vocab and a 1000-step budget there is no instability to suppress (zero runs diverged), and the penalty simply pulls the model off the cross-entropy objective.

4.2 Interactions

interaction = [effect of A | B on] − [effect of A | B off]; **negative = synergy** (A helps more when B is present), positive = redundancy/antagonism.

A	B	interaction	reading
optimizer	qk_norm	−0.158	strong synergy
layerscale	value_residual	−0.071	synergy
softcap	qk_gain	−0.007	∼none
z_loss	softcap	+0.006	∼none
z_loss	qk_gain	+0.003	∼none

The Muon×QK-norm interaction is the headline. The 2 × 2 cell means make it concrete:

	QK-norm off	QK-norm on
AdamW	1.987	1.929
Muon	1.964	**1.749**

Alone, Muon buys ≈0.023 nats and QK-norm ≈0.058 nats; together they buy ≈0.238 — roughly three times the sum of the parts. Muon orthogonalizes the body weight updates, and QK-norm bounds the scale of the query/key projections feeding attention; the data say these two stabilizations are complementary, and that **most of the "modern recipe" advantage is concentrated in this single pair.** LayerScale and value residuals show the same pattern more weakly: each gates or re-routes the residual stream, and they reinforce each other (−0.071).

4.3 Run rankings and the kitchen-sink trap

The best individual configurations are all Muon + QK-norm based:

rank	**configuration**	**final val loss**
1	Muon, all on **except z-loss**	**1.651** (seed mean 1.653)
4	Muon, full recipe (all on)	1.669 (seed mean 1.670)
19	Muon + QK-norm only	1.747
22	Muon, all on except QK-norm	1.781
–	AdamW baseline (all off)	1.987
–	AdamW full recipe	1.847

The top three runs (all three seeds) are the full recipe *with z-loss removed*, beating the kitchen-sink configuration by ≈0.018 nats — consistent with the main-effect sign of z-loss. Dropping QK-norm from the full recipe (rank 22, 1.781) is the single most damaging leave-one-out, again confirming QK-norm as the load-bearing component. Note also that the full recipe under **AdamW** (1.847) never reaches even the *worst* Muon recipe run, underscoring how much of the gain flows through the optimizer×QK-norm channel.

End to end, the recipe moves the model from 1.987 (baseline) to 1.651 (best), a 0.336-nat reduction, of which the Muon×QK-norm pair accounts for the large majority.

5 Discussion

A practitioner's shortlist. If you must port only part of this stack, port Muon and QK-norm *together* — separately they underwhelm, together they deliver. LayerScale and value residuals are a worthwhile second tier. QK-gain and softcap can be dropped with no measurable cost at this scale, and z-loss should be dropped outright unless a longer schedule or larger vocab reintroduces the instability it is meant to control.

Why z-loss flips sign. z-loss and softcap both exist to tame logit blow-up over long training. This study's regime (short budget, small vocab, zero divergences) is exactly the one where that insurance is unnecessary, so it shows up as pure cost (z-loss) or pure no-op (softcap). The result is a reminder that "stability" components should be judged against an actually-unstable baseline, not folded into a recipe by default.

Scope and limitations.

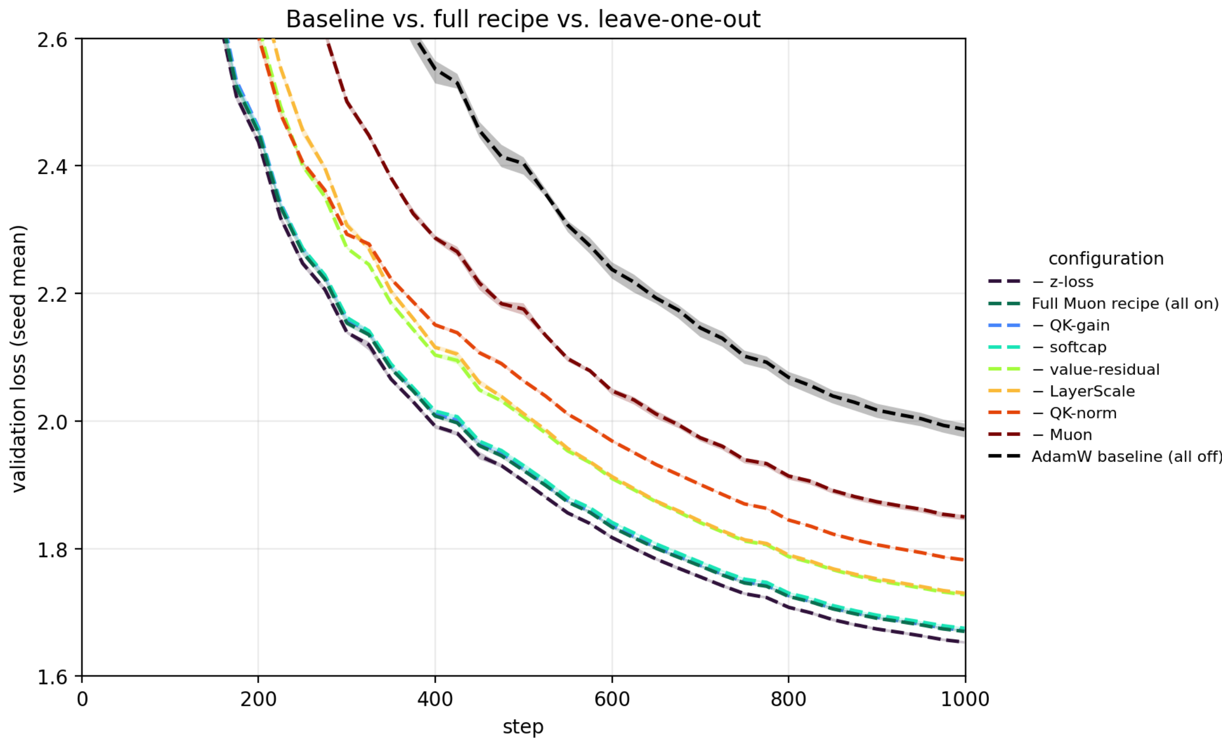

Figure 1: Validation-loss curves, one line per configuration (mean over 3 seeds, with a shaded min–max band; the bands are narrow because seed spread, std ≈0.003–0.008 nats, is small next to the between-config gaps) for the two anchors — the all-off AdamW baseline (dashed black) and the all-on full Muon recipe (green) — and the leave-one-out runs (full recipe with one component removed, labelled by the dropped component). The full recipe separates from the baseline early and holds the gap throughout; among the leave-one-outs, dropping Muon or QK-norm is by far the most damaging, while dropping z-loss slightly *improves* on the full recipe.

- **Scale.** One model size (∼98M), one corpus (TinyStories — simple, low-entropy text), one short budget (30.72M tokens). The *signs* of the large effects are likely robust, but the inert/harmful verdicts for z-loss and softcap are the components most likely to flip at larger vocab or longer schedules — precisely their design regime. These are the priority for a follow-up.
- **Fractional design.** Three- and higher-way interactions are not estimated; main effects and the five targeted pairs are. The strong Muon×QK-norm synergy means single-factor numbers should be read as context-dependent, not additive.
- **Single metric.** Final validation loss only — no downstream-task or generation-quality evaluation, and no measurement of wall-clock cost (Muon and the extra parameters add a small per-step overhead, visible in the logged `wall_time_s` but not analyzed here).
- **Seeds.** Three seeds give a usable variance estimate; the per-factor std columns show the large effects sit well outside seed noise, while QK-gain and softcap sit inside it.

6 Conclusion

The "modern transformer recipe" is not monolithic. On a controlled TinyStories ablation, its benefit concentrates almost entirely in **Muon + QK-norm acting together**, with a smaller LayerScale + value-residual contribution and a trailing set of components that are inert (QK-gain, softcap) or mildly counterproductive (z-loss) at this scale. The single best configuration is the full recipe with z-loss removed. The broader lesson is methodological: stack ingredients earn their place against matched, multi-seed pairs and explicit interaction tests — not by accumulation.

A Reproduction

```
python DataScripts/prepare_data.py          # one-time tokenization
python DataScripts/run_matrix.py --seeds 3  # 66 runs, resumable
python DataScripts/analyze_results.py       # tables + figures
```

Per-run artifacts: `DataOutput/runs/<run_id>/{config.json, log.jsonl, summary.json}`. Aggregated tables and loss-curve figures: `DataOutput/analysis/{results.csv, main_effects.csv, interactions.csv, summary.md, loss_curves*.png}`. All 66 runs completed; 0 diverged.